" OrbitMind AI– Intelligent Productivity & Study Companion"

by

Md. Imroz Newaz

CSE 11220321052

Zeniya Naznin

CSE 11230121138

Fatema Parvin Kanta

CSE 11230121192

Md. Mahfuzur Rahman Hridoy

CSE 11230121186

WEEK 08 ASSIGNMENT

Course Title: Software Development III: AI-Based Software Project Development

Course Code: CSE 4104

Section: 7D | Spring 2026

Submitted to

Md. Riaz Mahmud

Assistant Professor

Department of Computer Science and Engineering

Northern University of Business and Technology, Khulna

Khulna 9100, Bangladesh

CONTENTS

Title Page 1

Contents 2

Figures 3

CHAPTER I – Team Information 4

1.1 Team Information Sheet 4

1.1.1 Team Members 4

CHAPTER II – AI Integration Report 5-9

2.1 Selected AI Platform & Models Used 5

2.2 AI Features Implemented & Problem-Solving Capability 5

2.3 AI Workflow Design 5-6

2.4 Prompt Engineering Strategy & Examples 6-7

2.5 AI Response Handling & Validation Strategy 7

2.6 Current Development Progress 8

2.7 GitHub Repository Link 8-9

CHAPTER III – Screenshots 10-16

3.1 AI Feature Interface: The AI Study Planner 10

3.2 AI Responses: Structured JSON Rendering 10-11

3.3 AI Interaction: Coach Chatbot (Intent Recognition) 11-12

3.4 AI Feature: Note Summarizer 13

3.5 AI Feature: Productivity Audit (Dashboard) 14

3.6 Prompt Examples & Backend Integration (Code) 14-15

3.7 AI Response Handling (Code) 15-16

3.8 Maintained GitHub Repository 16

FIGURES

2.3 Figure: AI Workflow Design 6

3.1 Figure: Planner interface gathering user constraints before dispatching the Gemini API. 10

3.2 Figure: The frontend dynamically rendering the heavily structured JSON response returned by the AI model. 11

3.3 Figure: The AI Coach Chatbot successfully interpreting conversational intent and auto-generating a scheduled task. 12

3.4 Figure: The AI Note Summarizer extracting key concepts from raw academic text. 13

3.5 Figure: The Productivity Audit engine calculating burnout risk based on upcoming deadlines. 14

3.6 Figure: Our backend architecture demonstrating strict System Prompt engineering to enforce JSON output. 15

3.7 Figure: The implementation of multi-model failovers and API error handling within our Express service. 15

3.8 Figure: Maintained GitHub repository documenting continuous AI integration commits. 16

CHAPTER I

1.1 Team Information Sheet

Field	Details
Team Name	CSE4104-7D-T04
Section	7D
Project Title	OrbitMind AI — Smart Productivity & Study Assistant
Team Leader	Md. Imroz Newaz (ID: 11220321052)

1.1.1 Team Members

Member Name	Student ID	Role
Md. Imroz Newaz	11220321052	Team Leader & Database Manager
Zeniya Naznin	11230121138	Frontend Developer
Fatema Parvin Kanta	11230121192	Backend Developer
Md. Mahfuzur Rahman Hridoy	11230121186	AI Integration Lead

CHAPTER II

2.1 Selected AI Platform & Models Used

Our team selected **Google Gemini** as our core AI platform because of its rapid inference speed, excellent natural language understanding, and native JSON output capabilities. To ensure high availability, we implemented a multi-model fallback architecture in our backend that iterates through gemini-2.5-flash, gemini-2.5-pro, gemini-2.0-flash, and gemini-1.5-flash.

2.2 AI Features Implemented & Problem-Solving Capability

We integrated four distinct AI modules to directly solve student productivity problems:

- AI Coach Chatbot: Acts as a 24/7 academic mentor, answering questions and intelligently extracting user intent to automatically schedule study tasks.
- AI Study Planner: Solves the problem of exam preparation paralysis by generating highly structured, day-by-day JSON roadmaps based on subjects and exam dates.
- AI Note Summarizer: Condenses long, complex lecture transcripts into easily digestible bullet points and key concepts.
- Productivity Audit Engine: Analyzes the user's pending tasks and deadlines to calculate a burnout risk score and provide personalized scheduling recommendations.

2.3 AI Workflow Design

The AI operations within our system follow a strict, secure workflow**:**

User Input (Frontend) → Secure Proxy (Backend API) → Prompt Assembly (Utils) → Google Gemini API → Raw Text Response → JSON Parsing & Validation (Backend) → Database Persistence → Formatted Display (Frontend). By keeping the API keys strictly on the Node.js backend using *.env* variables, we ensure zero credential exposure on the client-side.

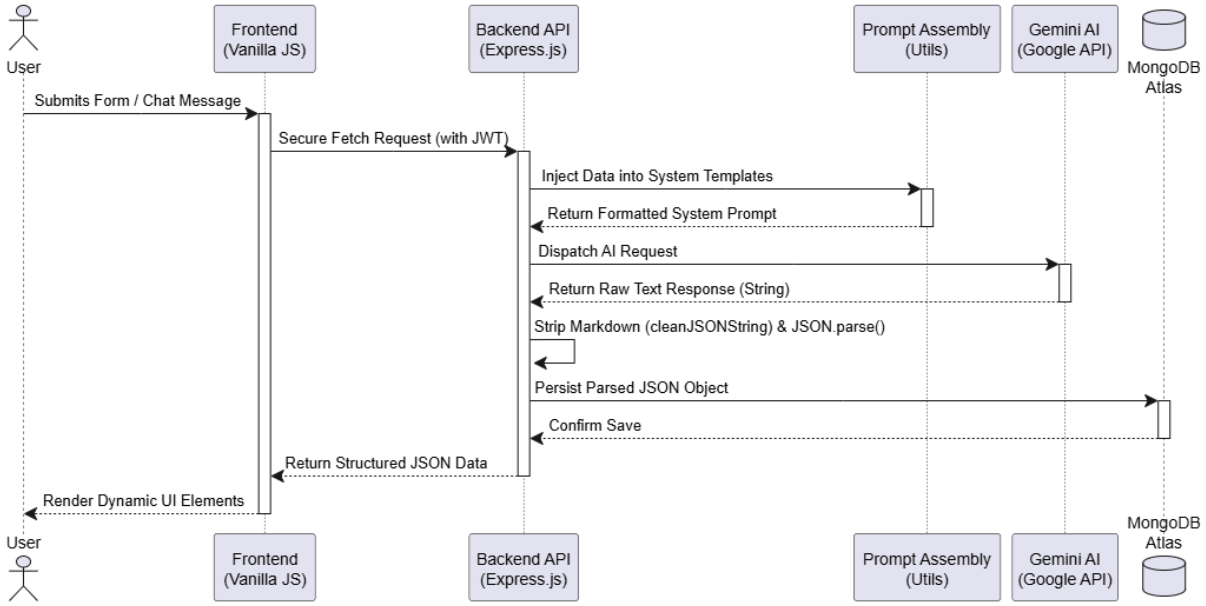


Figure 2.3: AI Workflow Design

2.4 Prompt Engineering Strategy & Examples

We utilized rigorous System Prompting combined with Few-Shot constraints to force the LLM to output valid JSON structures, preventing malformed UI rendering. We tailored distinct prompts for all four of our AI modules.

Example 1: AI Study Planner (Data Structuring)

- **System Prompt:** "Create a detailed, highly structured, student-friendly study plan. Subject: [Subject]. Topics to cover: [Topics]. Respond ONLY with a valid JSON object matching the following structure: { 'estimatedCompletionTime': '...', 'weeklyRoadmap': [...] }"
- **Expected Output:** A perfectly formatted JSON roadmap array that our backend natively maps to Mongoose subdocuments without manual string manipulation.

Example 2: AI Coach Chatbot (Intent Recognition & Task Extraction)

- **System Prompt:** "You are OrbitMind Coach. Detect if the user wants to add, schedule, or create a task. If they want to create a task, extract the details (title, description, deadline). Always respond in JSON."
- **User Prompt:** "Add a task to review CS tomorrow at 4pm"

- **Expected Output:** {"reply": "I have scheduled your review session!", "taskToCreate": { "title": "Review CS", "deadline": "2026-08-14" } }

Example 3: AI Note Summarizer (Text Condensation)

- **System Prompt:** "Summarize the following student notes. Extract key summaries, bullet points, and main concepts. Respond ONLY with a valid JSON object matching the following structure: { 'title': '...', 'summary': '...', 'bulletPoints': [...], 'keyConcepts': [...] }"

- **Expected Output:** A JSON object where long, dense lecture transcripts are broken down into a concise summary paragraph and an array of critical vocabulary concepts.

Example 4: Productivity Audit (Analytical Scoring)

- System Prompt: "Analyze this student's workload, pending tasks, and upcoming deadlines. Provide an audit assessing their current schedule and safety from burnout. Respond ONLY with a valid JSON object matching this structure: { 'productivityScore': 85, 'burnoutLevel': 'low', 'priorityAlerts': [...] }"

- Expected Output: An analytical JSON assessment that alerts the student if they have too many tasks due on the same day, helping them actively prevent academic burnout.

2.5 AI Response Handling & Validation Strategy

We designed our system to be highly fault-tolerant.

- **Invalid Responses/Empty Outputs:** Our backend utilizes a cleanJSONString() function to strip unexpected markdown formatting (e.g., ```*json* blocks) injected by the LLM before calling *JSON.parse()*.

- **Network Failures/Rate Limits:** If the Gemini API experiences a timeout, rate limit, or network failure, our try-catch blocks automatically fall back to the next available Gemini model.

- **Service Unavailability:** If all external API calls fail, our system smoothly injects a localized getMockResponse() module, returning simulated AI data so the application never crashes and the user experience remains uninterrupted.

2.6 Current Development Progress

As of Week 08, the core development of the OrbitMind AI platform is fully operational, marking the successful completion of our most critical milestone: end-to-end Artificial Intelligence integration. The application has evolved from static UI mockups into a cohesive, intelligent, and highly interactive academic assistant.

Backend & Architecture: Our Node.js and Express backend is completely stable. We have established secure JWT session management and successfully mapped all CRUD operations to our MongoDB Atlas cloud cluster. Our routing architecture strictly adheres to the MVC design pattern, ensuring that data flows cleanly and securely.

Frontend UI Integration: The Vanilla JavaScript frontend has been completely integrated with the backend. Dynamic data fetching is functioning perfectly, meaning the dashboard, task lists, and study plans update in real-time based on database state. We have also ensured the interface is fully responsive across desktop and mobile viewports, complete with comprehensive client-side form validation.

AI Integration Milestone: We have successfully integrated all four of our target AI modules (Study Planner, Coach Chatbot, Note Summarizer, and Productivity Audit). The Google Gemini API is securely accessed via our backend proxy, and we have fortified the system with advanced error handling. By implementing a failover loop that iterates through multiple Gemini models (e.g., falling back to gemini-1.5-flash if gemini-2.5-pro fails), we guarantee high availability. Furthermore, our strict prompt engineering completely eliminated malformed responses by forcing the LLM to output pure JSON.

Next Steps: With the frontend, backend, database, and AI components seamlessly communicating, our final phase will involve comprehensive system testing, performance optimization, and preparing the application for live deployment.

2.7 GitHub Repository Link

Our team continues to maintain an organized repository with meaningful, descriptive commit messages documenting our frontend development cycle.

Repository	Link
Official GitHub Repo	https://github.com/Mahfuz-Hridoy/cse4104-7d-t04-orbitmind-ai.git

CHAPTER III

3.1 AI Feature Interface: The AI Study Planner

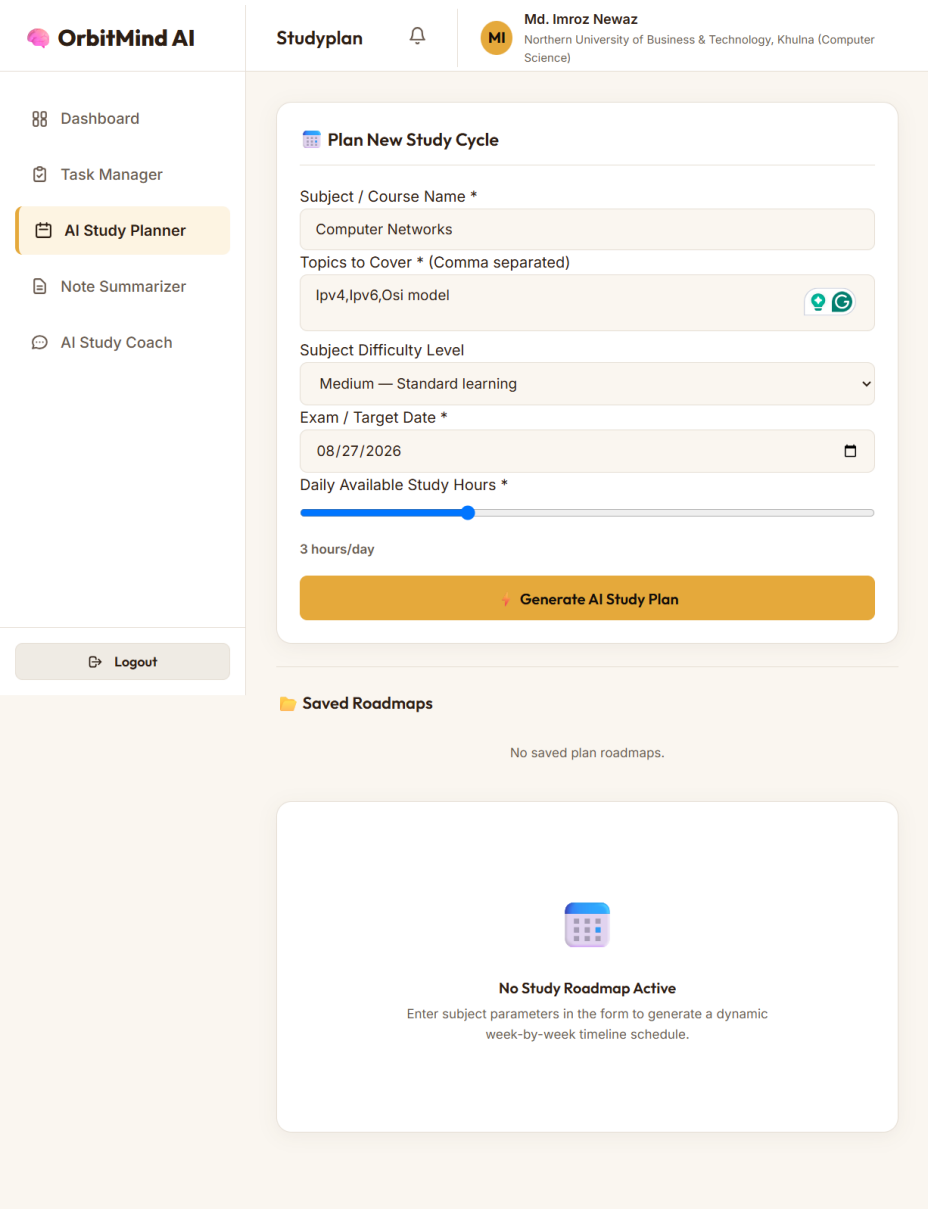


Figure 3.1: The AI Study Planner interface gathering user constraints before dispatching to the Gemini API.

Implementation Detail: This UI serves as the primary data collection point. We strictly control the inputs (Subject, Topics, Exam Date) to ensure the backend can construct a highly constrained, predictable prompt for the LLM.

3.2 AI Responses: Structured JSON Rendering (Study Planner Output)

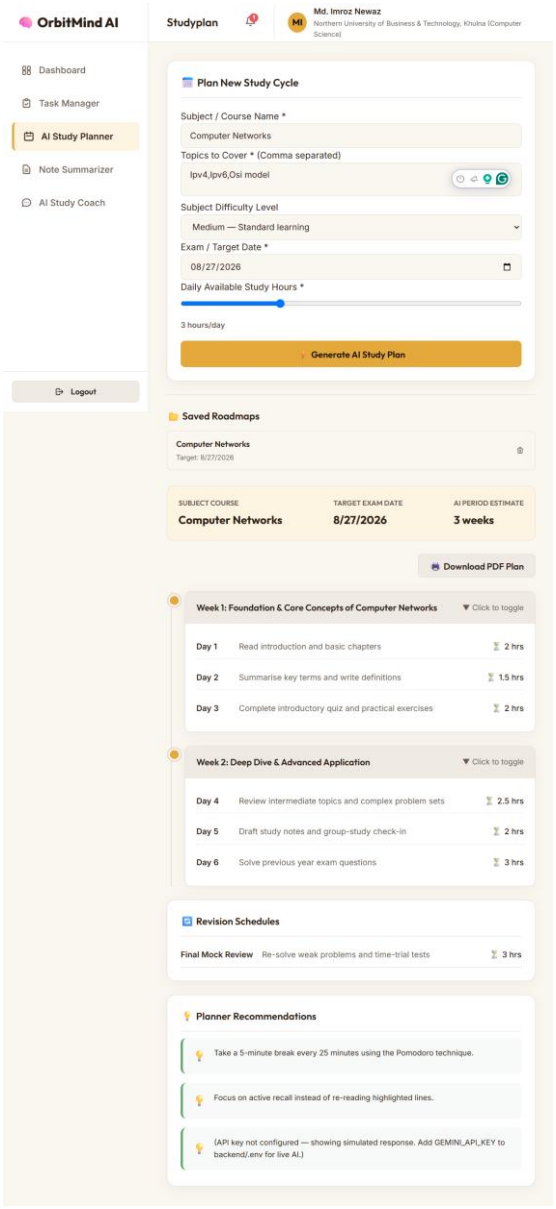


Figure 3.2: The frontend dynamically rendering the heavily structured JSON response returned by the AI model.

Implementation Detail: This proves our strict Prompt Engineering strategy was successful. The Gemini API returned a raw JSON string which our backend parsed and transmitted to the frontend, allowing the UI to render the academic roadmap dynamically without breaking the layout.

3.3 AI Interaction: Coach Chatbot (Intent Recognition)

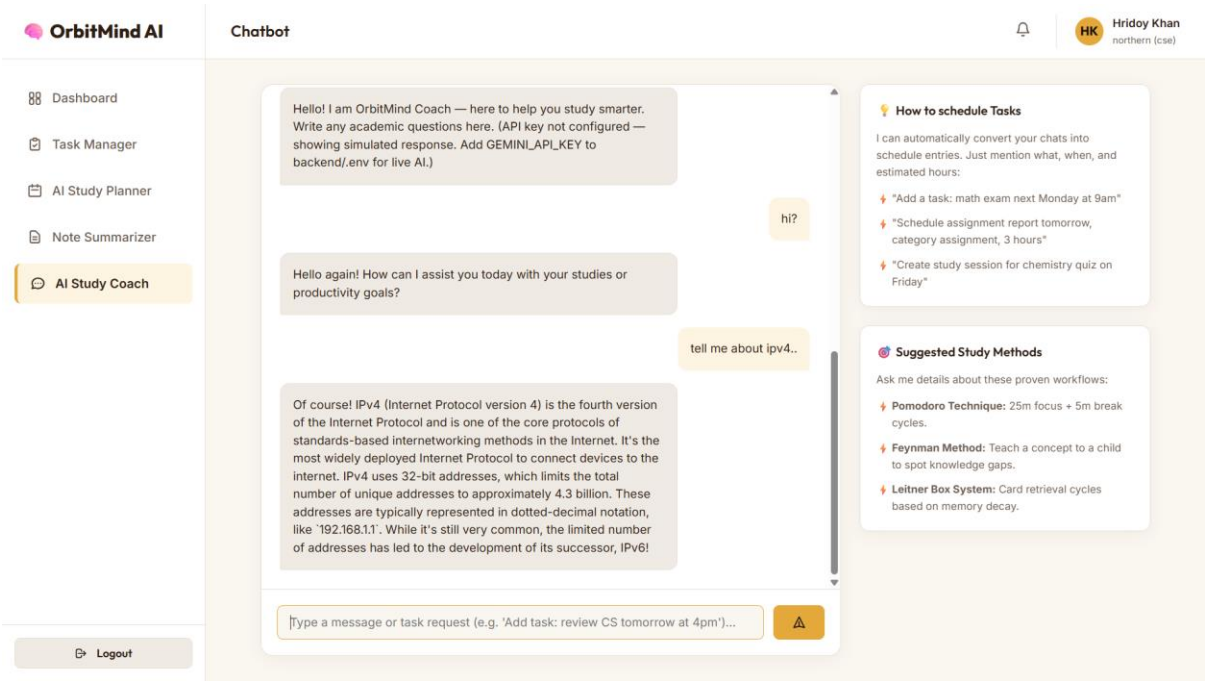


Figure 3.3: The AI Coach Chatbot successfully interpreting conversational intent and auto-generating a scheduled task.

Implementation Detail: This demonstrates advanced AI Interaction. The LLM acts not just as a text generator, but as an intent-recognition engine. It analyzed the user's conversational prompt, identified a task-creation intent, and returned a structured object that our backend intercepted to silently create a database entry.

3.4. AI Feature: Note Summarizer

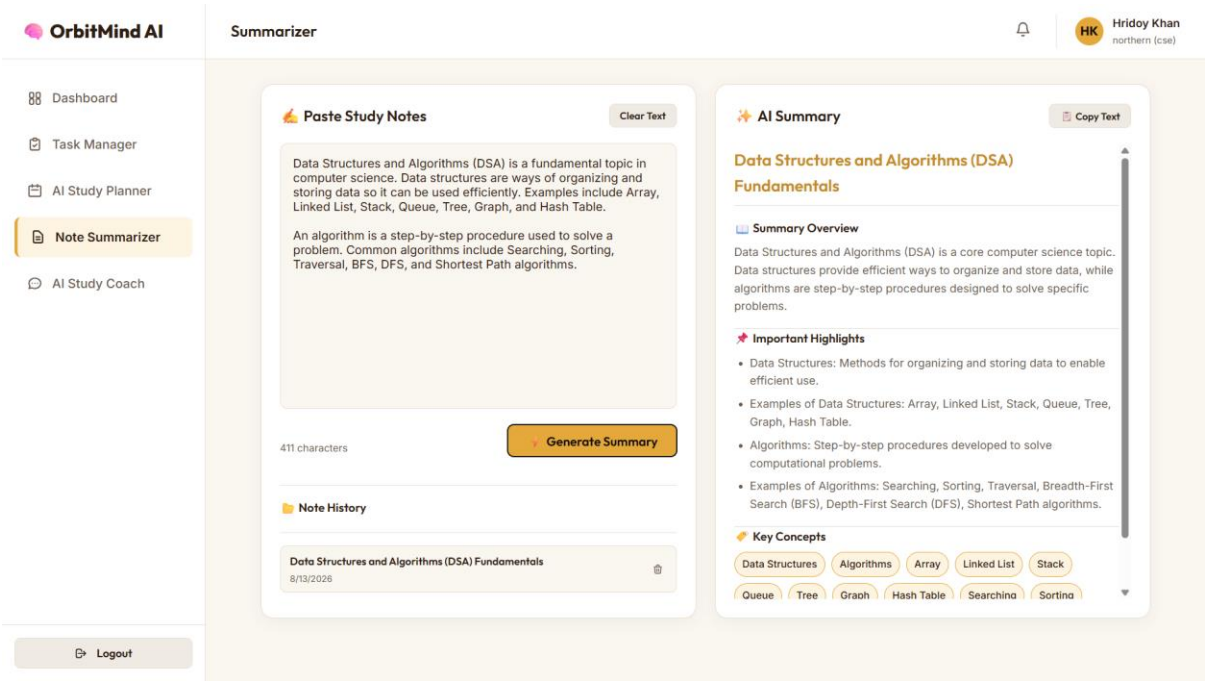


Figure 3.4: The AI Note Summarizer extracting key concepts from raw academic text.

Implementation Detail: This feature highlights our system's ability to handle large token inputs. The frontend submits raw, unstructured lecture transcripts, and the AI distills theinformation into concise arrays of key concepts and bullet points for rapid revision.

3.5 AI Feature: Productivity Audit (Dashboard)

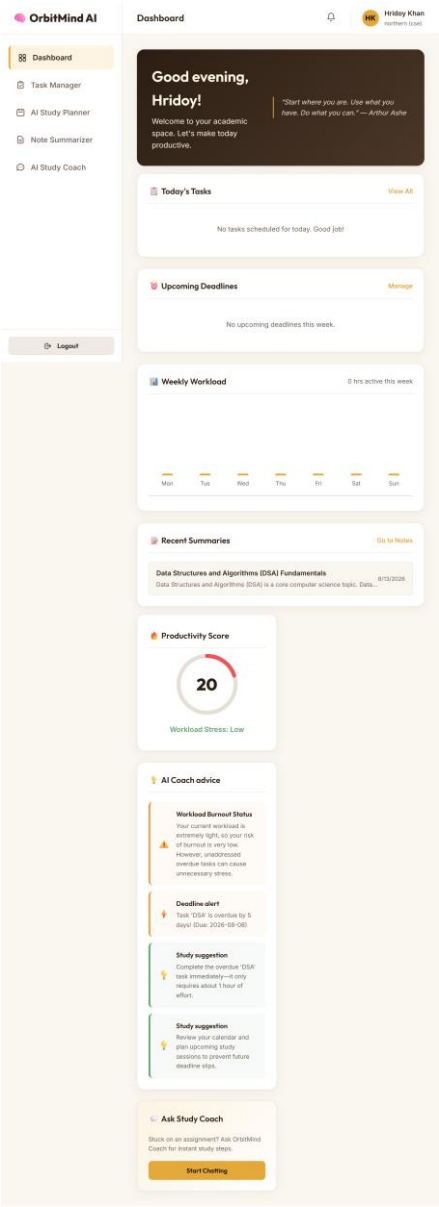


Figure 3.5: The Productivity Audit engine calculating burnout risk based on upcoming deadlines.

Implementation Detail: This module adds measurable value by solving a real-world problem: student burnout. The backend compiles the user's current task array, submits it to the AI, and receives an analytical productivity score with personalized scheduling alerts.

3.6 Prompt Examples & Backend Integration (Code)

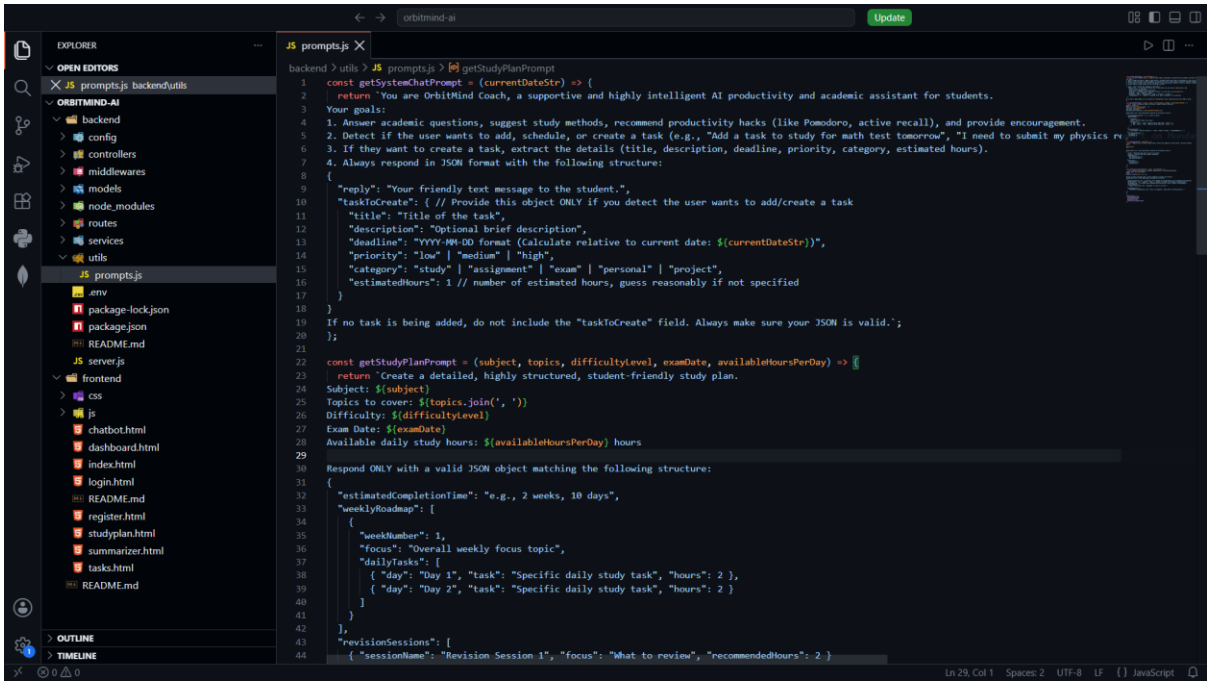


Figure 3.6: Our backend architecture demonstrating strict System Prompt engineering to enforce JSON output.

Implementation Detail: This code snippet highlights our Prompt Engineering methodology. We utilize Few-Shot prompting and explicit JSON formatting rules to ensure the Gemini model output is strictly constrained, completely eliminating the risk of unparseable frontend data.

3.7 AI Response Handling (Code)

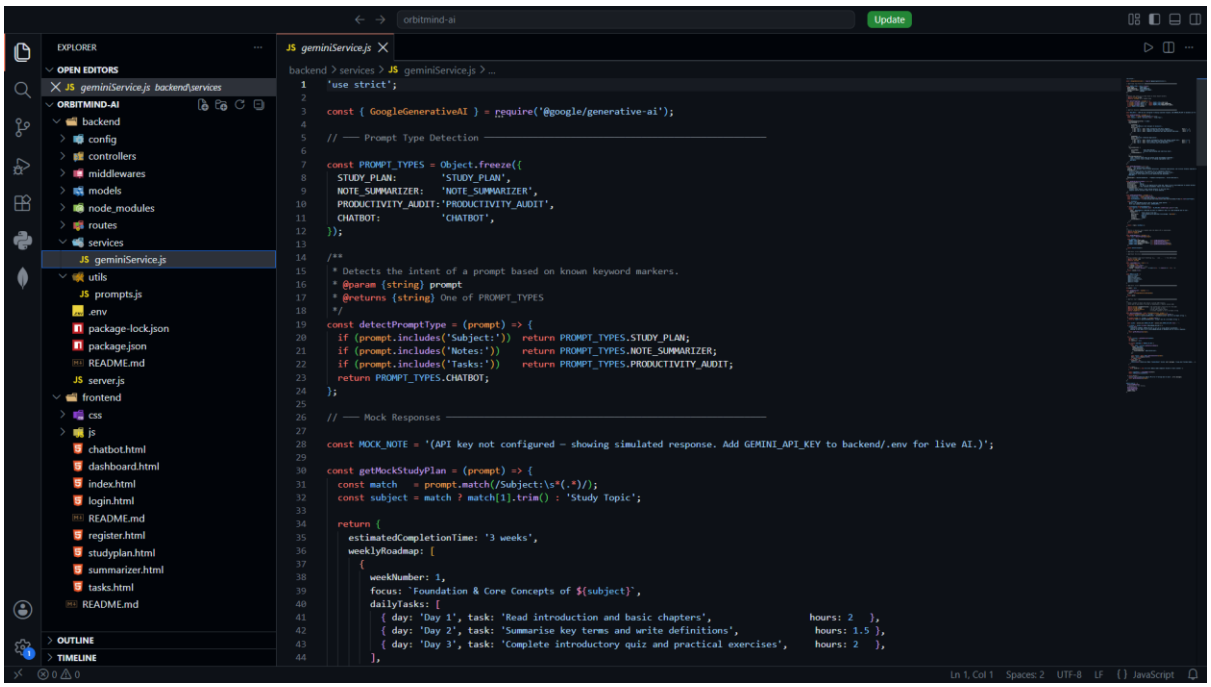


Figure 3.7: The implementation of multi-model failovers and API error handling within our Express service.

Implementation Detail: This proves our robust AI Response Handling strategy. The backend executes a failover loop across multiple Gemini models (MODELS_TO_TRY). If an API timeout, rate limit, or network failure occurs on the primary model, the system smoothly defaults to the next available endpoint, ensuring zero downtime.

3.8 Maintained GitHub Repository

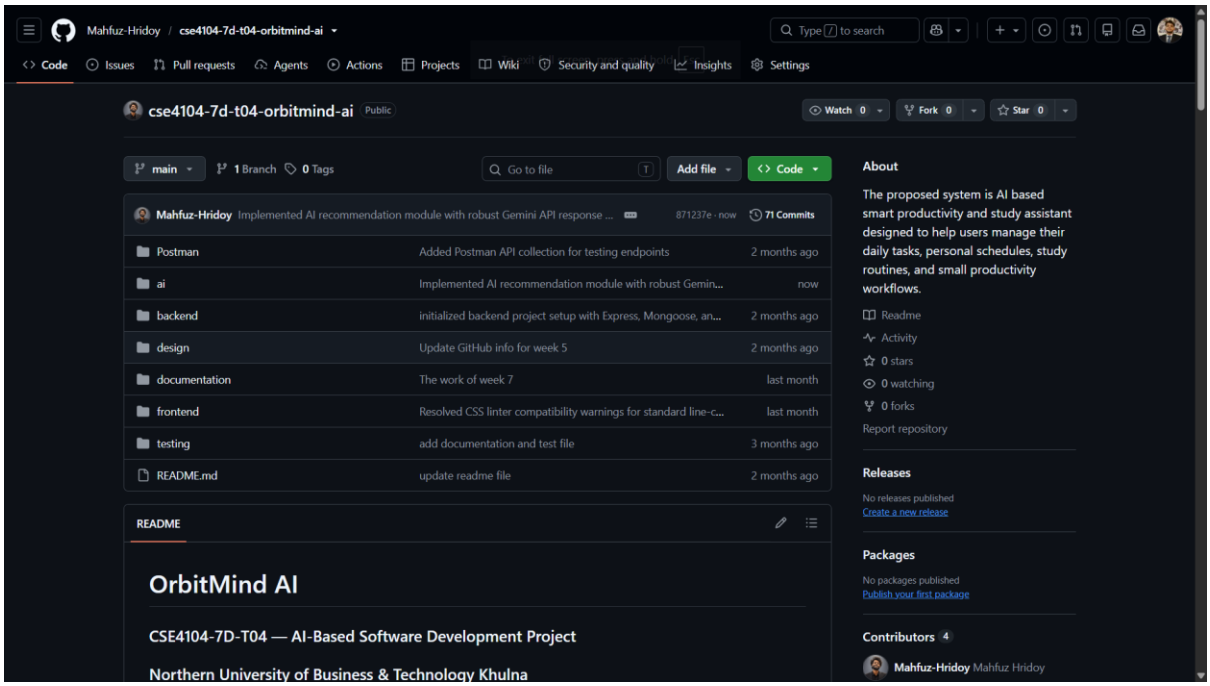


Figure 8: Maintained GitHub repository documenting continuous AI integration commits.

Implementation Detail: Our GitHub workflow strictly adheres to the recommended repository structure. We implemented dedicated directories for AI documentation and services, accompanied by meaningful commit messages that trace the evolution of our AI modules.